

## Contents

|                                                                                    |          |
|------------------------------------------------------------------------------------|----------|
| <b>Programming Project Phase 2 Rubric — RDT 2.2 over an Unreliable UDP Channel</b> | <b>1</b> |
| Requirements . . . . .                                                             | 1        |
| Phase 2(a): RDT 2.2 protocol implementation . . . . .                              | 1        |
| Phase 2(b): Packetization and reassembly . . . . .                                 | 1        |
| Phase 2(c): Required transfer scenarios . . . . .                                  | 2        |
| Phase 2(d): Performance evaluation and plots . . . . .                             | 2        |
| Deliverables and submission . . . . .                                              | 2        |
| Grading breakdown . . . . .                                                        | 3        |
| A) Design Document (required first) — 15 points . . . . .                          | 3        |
| B) RDT 2.2 Core Correctness — 30 points . . . . .                                  | 3        |
| C) Required Scenarios (Options 1–3) — 20 points . . . . .                          | 4        |
| D) Performance Evaluation & Plots — 15 points . . . . .                            | 4        |
| E) README & Reproducibility — 10 points . . . . .                                  | 4        |
| F) Repo, Contributions, and Demo Video — 10 points . . . . .                       | 4        |
| Grader notes template . . . . .                                                    | 4        |

## Programming Project Phase 2 Rubric — RDT 2.2 over an Unreliable UDP Channel

Total: 100 points

---

### Requirements

#### Phase 2(a): RDT 2.2 protocol implementation

- **R1:** Implement file transfer over UDP using **RDT 2.2** (Kurose/Ross, Section 3.4.1).
- **R2:** Use sequence numbers to detect duplicates.
- **R3:** Implement a checksum for error detection on DATA and ACK packets.
- **R4:** Implement ACK-based recovery (RDT 2.2 is NAK-free).
- **R5:** Sender and receiver behavior must match the RDT 2.2 sender/receiver FSM semantics.

#### Phase 2(b): Packetization and reassembly

- **R6:** Packetize the transfer file into a constant payload size per packet (example payload size: 1024 bytes).
- **R7:** Non-pipelined sending (send next packet only after the previous is acknowledged / retransmission logic completes).

- **R8:** Receiver must reassemble packets in order and deliver the full file to the application.
- **R9:** In the no-error mode, the output file must match the input file (byte-for-byte correctness preferred).

### Phase 2(c): Required transfer scenarios

You must demonstrate all three scenarios below:

- **R10:** Option 1 — No loss and no bit-errors.
- **R11:** Option 2 — ACK packet bit-error injection at the sender-side receive path (and correct recovery).
- **R12:** Option 3 — DATA packet bit-error injection at the receiver-side receive path (and correct recovery).

### Phase 2(d): Performance evaluation and plots

- **R13:** Measure end-to-end completion time for transferring the same file under each option.
- **R14:** Produce plots of completion time versus loss/error rate from 0% to 95% in increments of 5%.
- **R15:** Each plotted point must be an average of at least 5 runs at that rate.
- **R16:** Remove (or disable) debug/print logging during timing experiments.

### Plot requirements (what it should look like):

- **x-axis:** loss/error rate (%), using this list exactly:
  - `rates = [0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95]`
- **y-axis:** completion time in seconds (average of runs).

### Recommended data structure for averaging:

- Store raw runs in a matrix (list of lists) per option:
  - `times_option1[r][k]` = completion time for rate index `r` and run index `k`
  - shape: `len(rates) x 5` (or more runs)
- Then compute:
  - `avg_option1[r] = mean(times_option1[r])`
  - Same for Option 2 and Option 3
- Final plot:
  - three lines on the same chart (Option 1, Option 2, Option 3), all against the same `rates` x-axis.

### Deliverables and submission

- **R17:** Submit `DESIGN_DOCUMENT.pdf` or `DESIGN_DOCUMENT.md`.
  - If submitting on Blackboard, it must be PDF.

- If submitting via GitHub only, Markdown is allowed.
  - **R18:** Submit `README.pdf` or `README.md` with run instructions for all scenarios and how to reproduce plots.
  - **R19:** Submit the plots (and include the raw timing data file(s) if you generated them).
  - **R20:** Submit `contribution.txt` or `contribution.doc` (individual, confidential).
  - **R21:** Repository setup:
    - Use one shared GitHub repo for the team (private is fine).
    - The repo must include all code, docs, and plot scripts.
    - The instructor must be invited to this repo and have access (verify access before submission).
  - **R22:** Demo video:
    - Submit a short YouTube (unlisted) or video link showing:
      - \* Option 1 working end-to-end
      - \* Option 2 working end-to-end
      - \* Option 3 working end-to-end
      - \* A quick look at the generated plot(s)
  - **R23:** Academic integrity: submissions may be inspected using plagiarism detection software.
- 

## Grading breakdown

### A) Design Document (required first) — 15 points

- **15:** Clear, complete design submitted before major coding. Includes:
  - Sender/receiver logic aligned with the RDT 2.2 FSM
  - Packet format (seq number, checksum, payload, ACK format)
  - Error injection approach for Options 2 and 3
  - Basic test plan and validation steps
- **10:** Mostly complete but missing one key element
- **5:** Thin/unclear or looks written after implementation
- **0:** Not submitted

### B) RDT 2.2 Core Correctness — 30 points

- **30:** Correct behavior across normal operation and retransmissions; no protocol logic issues
- **24:** Mostly correct with minor issues (rare edge cases or minor logic inconsistencies)
- **16:** Partially correct (protocol breaks in at least one common case)
- **0–8:** Not working / cannot be demonstrated

**C) Required Scenarios (Options 1–3) — 20 points**

- **20:** All three options implemented and demonstrated correctly
- **15:** Two options correct
- **10:** One option correct
- **0:** No correct scenario handling

**D) Performance Evaluation & Plots — 15 points**

- **15:** Correct x/y axes, full rate range, averaging done correctly; plots are readable and labeled
- **10:** Minor issues (missing a few points, unclear labels, incomplete averaging evidence)
- **5:** Incomplete or incorrect methodology
- **0:** Missing

**E) README & Reproducibility — 10 points**

- **10:** Clear instructions to run all options, reproduce plots, and interpret outputs
- **6:** Instructions present but missing steps or ambiguous
- **3:** Minimal
- **0:** Missing

**F) Repo, Contributions, and Demo Video — 10 points**

- **10:** Repo access verified, meaningful commit history, contribution file submitted, and demo video complete
- **6:** Repo and contributions present but weak history and/or video is incomplete
- **3:** Minimal evidence of collaboration and/or unclear demo
- **0:** Missing repo access / missing contribution file / missing video

---

**Grader notes template**

| Item                                | Max Pts    | Earned | Notes / Deductions |
|-------------------------------------|------------|--------|--------------------|
| A (Design Document)                 | 15         |        |                    |
| B (RDT 2.2 Core Correctness)        | 30         |        |                    |
| C (Options 1–3)                     | 20         |        |                    |
| D (Performance & Plots)             | 15         |        |                    |
| E (README & Reproducibility)        | 10         |        |                    |
| F (Repo, Contributions, Demo Video) | 10         |        |                    |
| <b>TOTAL</b>                        | <b>100</b> |        |                    |